TI-2400 Processamento de Linguagem Natural

ANTONIO CARVALHO - TREINAMENTOS

Numpy

O que é Numpy

NumPy é uma biblioteca Python voltada à computação numérica, cujo principal objeto é o **ndarray** um array multidimensional homogêneo capaz de armazenar e processar grandes volumes de dados de forma eficiente.

Esta biblioteca é fundamental para computação científica com Python

NumPy documentation — NumPy v1.25 Manual

Diferenças entre o Numpy e as listas sequenciais em Python

- As matrizes do Numpy possuem um tamanho fixo já no momento de criação.
- Diferentemente das listas Python, um **ndarray** não cresce dinamicamente. Operações que acrescentam ou removem elementos normalmente criam um novo ***array***.
- Os elementos da matriz precisam ser todos do mesmo tipo de dado.
- As matrizes facilitam operações matemáticas em grandes volumes de dados.

What is NumPy? — NumPy v1.25 Manual

Dimensões e nomenclaturas

```
import numpy as np
a = np.array([10, 20, 30])          # 1 dimensão
b = np.array([[1, 2], [3, 4]])      # 2 dimensões
c = np.zeros((2, 3, 4))             # 3 dimensões
```

Dimensões e a nomenclatura usada:

Número de Dimensões	Nomenclatura utilizada
0	**Escalar**
1	**Vetor**
2	**Matriz**
3 ou mais	**Tensor** ou **Array multidimensional**

Arrays, Propriedades básicas

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]], dtype=np.float32)
print(a.ndim, a.shape, a.dtype)
```

1. Arrays podem conter qualquer número de dimensões incluindo zero (um escalar).
2. Cada **ndarray** possui um **dtype**, que determina como seus elementos são representados na memória.: np.uint8, np.int64, np.float32, np.float64
3. Arrays são densas. Todos os elementos do array existem e são do mesmo tipo.

Arrays, criação

np.ones, np.zeros

np.arange

np.linspace

np.zeros_like, np.ones_like

np.random.random

Arrays, criação

np.ones, np.zeros

np.arange

np.linspace

np.zeros_like, np.ones_like

np.random.random

```
>>> np.ones((3,5),dtype=np.float32)
array([[ 1.,  1.,  1.,  1.,  1.],
       [ 1.,  1.,  1.,  1.,  1.],
       [ 1.,  1.,  1.,  1.,  1.]], dtype=float32)
```

```
>>> np.zeros((6,2),dtype=np.int8)
array([[0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0],
       [0, 0]], dtype=int8)
```

Arrays, criação

np.ones, np.zeros

np.arange

np.linspace

np.zeros_like, np.ones_like

np.random.random

```
>>> np.arange(1334,1338)
array([1334, 1335, 1336, 1337])
```

Arrays, criação

np.ones, np.zeros

np.arange

np.linspace

np.zeros_like, np.ones_like

np.random.random

```
>>> A = np.linspace(1, 4, 10)
>>> A
array([1.        , 1.33333333, 1.66666667, 2.        , 2.33333333,
       2.66666667, 3.        , 3.33333333, 3.66666667, 4.        ])
>>> B = np.linspace(1, 100, 5)
>>> B
array([  1.  ,  25.75,  50.5 ,  75.25, 100.  ])
```

Na função **linspace** é informado **quantos** elementos internos, entre o start e stop, são desejados

Arrays, criação

np.ones, np.zeros

np.arange

np.linspace

np.zeros_like, np.ones_like

np.random.random

```
>>> a = np.ones((2,2,3))
>>> b = np.zeros_like(a)
>>> print(b.shape)
```

Arrays, criação

np.ones, np.zeros

np.arange

np.linspace

np.zeros_like, np.ones_like

np.random.random

```
>>> np.random.random((3, 4))
array([[0.13925491, 0.52204366, 0.46375872, 0.56601159],
       [0.17719879, 0.1342495 , 0.36311068, 0.95859705],
       [0.15152296, 0.93181784, 0.05336024, 0.89419866]])
>>> rng = np.random.default_rng( 50 )
>>> A = rng.random((3, 4))
>>> A
array([[0.78742269, 0.83366933, 0.54790446, 0.9734491 ],
       [0.23683356, 0.64692334, 0.06502607, 0.55557761],
       [0.40235748, 0.2659658 , 0.9866823 , 0.39731767]])
>>> B = rng.integers(1, 10, size = (4, 5))
>>> B
array([[2, 3, 6, 2, 9],
       [2, 3, 9, 6, 6],
       [5, 9, 5, 1, 8],
       [6, 2, 4, 5, 4]])
```

Arrays, criação

```
array = np.array([[0,1,2],[2,3,4]])
[[0 1 2]
 [2 3 4]]

array = np.zeros((2,3))
[[0. 0. 0.]
 [0. 0. 0.]]

array = np.ones((2,3))
[[1. 1. 1.]
 [1. 1. 1.]]
```

```
array = np.eye(3)
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

array = np.arange(0, 10, 2)
[0 2 4 6 8]

array = np.random.randint(0, 10, (3,3))
[[6 4 3]
 [1 5 6]
 [9 8 5]]
```

Arrays, conversão

np.astype

```
>>> A
array([[ 4670.5,  4670.5,  4670.5],
       [ 4670.5,  4670.5,  4670.5],
       [ 4670.5,  4670.5,  4670.5],
       [ 4670.5,  4670.5,  4670.5],
       [ 4670.5,  4670.5,  4670.5]], dtype=float32)
>>> print(A.astype(np.uint16))
[[4670 4670 4670]
 [4670 4670 4670]
 [4670 4670 4670]
 [4670 4670 4670]
 [4670 4670 4670]]
```

Arrays, combinação

np.concatenate

np.stack

```
>>> A = np.ones((4,1))
>>> B = np.zeros((4,2))
>>> np.concatenate([A,B], axis=1)
array([[ 1.,  0.,  0.],
       [ 1.,  0.,  0.],
       [ 1.,  0.,  0.],
       [ 1.,  0.,  0.]])
```

Arrays, combinação

np.concatenate

np.stack

```
>>> A = np.ones((4,3))
>>> A
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
>>> B = np.zeros((4,3))
>>> B
array([[0., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.],
       [0., 0., 0.]])
>>> np.stack([A, B])
array([[[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]],

       [[0., 0., 0.],
        [0., 0., 0.],
        [0., 0., 0.],
        [0., 0., 0.]]])
>>> np.stack([A, B], axis=1)
array([[[1., 1., 1.],
        [0., 0., 0.]],

       [[1., 1., 1.],
        [0., 0., 0.]],

       [[1., 1., 1.],
        [0., 0., 0.]],

       [[1., 1., 1.],
        [0., 0., 0.]]])
```

De maneira diferente do **np.concatenate** (que junta os dados em um eixo que já existe), o **np.stack** introduz uma dimensão inédita onde os arrays são encaixados.

Para seu funcionamento, os **arrays** precisam ter os mesmos **formatos**

Broadcasting

O termo ***broadcasting*** descreve como o NumPy trata ***arrays*** com formatos diferentes durante operações aritméticas. Sujeito a certas restrições, o ***array*** menor é "propagado" (***broadcasted***) sobre o ***array*** maior, de modo que ambos passem a ter formatos compatíveis.

O ***broadcasting*** oferece um meio de vetorizar operações com ***arrays***, permitindo que a iteração ocorra em C, em vez de em Python.

Operações elemento a elemento exigem ***arrays*** com formas iguais ou compatíveis pelas regras de ***broadcasting***.

```
a = np.array([10, 20, 30])
a + 5      # [15 25 35]


notas = np.array([     [7, 8, 9],
                       [6, 5, 8],
                       [9, 8, 10] ])
bonus = np.array([1, 0, 0.5])
notas + bonus
```

Arrays, problemas

Precisam ser densas, sem buracos

Precisam ser de um único tipo

Operações elemento a elemento exigem arrays com formas iguais ou compatíveis pelas regras de ***broadcasting***.

```
>>> np.ones([7,8]) + np.ones([9,3])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: operands could not be broadcast together
 with shapes (7,8) (9,3)
```

Exercícios

Em Processamento de Linguagem Natural, textos precisam ser convertidos para representações numéricas antes de serem utilizados por algoritmos de Machine Learning. Considere que estamos preparando dados de um pequeno conjunto de documentos.

1. Importe a biblioteca NumPy e crie um array contendo os identificadores de 10 palavras de um vocabulário, numeradas de 1 até 10.

   Exemplo:

   1 → casa

   2 → carro

   3 → escola...

   10 → computador

2. Em modelos de PLN, cada palavra pode ser representada por um vetor numérico chamado embedding. Considere um conjunto contendo 300 palavras, sendo que cada palavra será representada por um vetor com 200 características numéricas.

Formato

```
a = np.array([1,2,3,4,5,6])

a = a.reshape(3,2)

a = a.reshape(2,-1)

a = a.ravel()
```

1. Número total de elementos não pode ser modificado.
2. Utilize -1 para o **NumPy** inferir automaticamente o tamanho de uma das dimensões.
3. **Row-major** por padrão. Significa que por padrão, o **NumPy** percorre os valores linha por linha (ordem C).

Valores retornados

Funções do Numpy retornam ambas **Views** e **Copias**

As **Views** compartilham os dados com o Array original. Ao alterar os ítens em uma **View** modificam os valores no Array original.

A documentação do numpy (https://numpy.org/doc/stable/reference/) mostra quais funções retornam **Views** ou **Cópias**

np.copy faz uma cópia de maneira explicita.

Exemplo:

```
a = np.array([10, 20, 30, 40])
b = a               # Mesmo objeto
c = a[1:3]          # Outro objeto, mas compartilha os dados
d = a.copy()        # Dados completamente idependentes
```

Transposição

```
a = np.arange(10).reshape(5,2)  # Não transpõem a matriz
a = a.T
a = a.transpose((1,0))
```

Em **arrays 2D**, **.T** troca linhas por colunas. Em arrays com mais dimensões, **.T** inverte a ordem dos eixos.

Salvando e carregando arrays

```
np.savez('data.npz', a=a)
data = np.load('data.npz')
a = data['a']
```

1. Arquivos NPZ podem conter múltiplos Arrays

Exercícios

1. Crie um array NumPy com 12 elementos e o redimensione em uma matriz 3x4. Em seguida, realize a transposição dessa matriz.

Operadores matemáticos

Operações artiméticas são elementares

Operadores lógicos retornam uma Array booleana

Operações **in-place** "locais" modificam o Array

```
>>> a
array([1, 2, 3])
>>> b
array([ 4,  4, 10])
>>> a * b
array([ 4,  8, 30])
```

O operador **(*) não é multiplicação matricial**, é uma **multiplicação elemento a elemento**.

Operadores matemáticos

Operações artiméticas são elementares

Operadores lógicos retornam uma Array booleana

Operações **in-place** "locais" modificam o Array

```
>>> a
array([[ 0.93445601,  0.42984044,  0.12228461],
       [ 0.06239738,  0.76019703,  0.11123116],
       [ 0.14617578,  0.90159137,  0.89746818]])
>>> a > 0.5
array([[ True, False, False],
       [False,  True, False],
       [False,  True,  True]], dtype=bool)
```

Operadores matemáticos

Operações artiméticas são elementares

Operadores lógicos retornam uma Array booleana

Operações in-place "locais" modificam o Array

```
>>> a
array([[ 4, 15],
       [20, 75]])
>>> b
array([[ 2,  5],
       [ 5, 15]])
>>> a /= b
>>> a
array([[2, 3],
       [4, 5]])
```

Operadores matemáticos

Ao combinar diferentes tipos numéricos, o NumPy aplica regras de promoção de **dtype** para determinar o tipo do resultado.

uint64 + uint16 => uint64

float32 / int32 => float32

Exemplo:

```
a = np.array([1], dtype=np.int32)
b = np.array([2.5], dtype=np.float64)
c = a + b
print(c.dtype)     # float64
```

Exercícios

1. Crie dois arrays NumPy de mesma forma e realize as quatro operações aritméticas básicas (+, -, *, /) entre eles.

Matemática, funções universais

Também chamadas de **ufuncs**

Examples:

- np.exp
- np.sqrt
- np.sin
- np.cos
- np.isnan

```
>>> a
array([[ 1,  4],
       [ 9, 16],
       [25, 36]])
>>> np.sqrt(a)
array([[ 1.,  2.],
       [ 3.,  4.],
       [ 5.,  6.]])
```

Indexação

```
x[0,0]    # Elemento superior esquerdo
x[0,-1]   # primeira linha, última coluna
x[0,:]    # primeira linha (várias células)
x[:,0]    # primera coluna (várias células)
```

Nota:

- Indices Multi-dimensionais são separados por virgula (como uma tuple)

Indexação

```
I[1:-1,1:-1]        # selecione tudo, exceto a
borda de um pixel

I = I[:,:,::-1]     # troca ordem de canal

I[I<10] = 0         # definir valores menores do
que 10 para 0

I[[1,3], :]         # seleciona a 2ª e a 4ª linha
```

1. Fatias do array são **Views**. Modificar um slice, altera o array original.

Máscara booleana

```
idades = np.array([18, 25, 16, 42, 14])
mascara = idades >= 18
print(mascara)                  # [ True  True False  True False]
print(idades[mascara])          # [18 25 42]
```

Fatias do array

Sintaxe: inicio:fim:passo

```
a = np.arange(10)
a[:3] # indices 0, 1, 2
a[-3:] # indices 7, 8, 9
a[3:8:2] # indices 3, 5, 7
a[4:1:-1] # indices 4, 3, 2
```

Fatias do array

```
arr = np.arange(10)
print(arr)                # [0 1 2 3 4 5 6 7 8 9]

arr_slice = arr[5:8]
print(arr_slice)          # [5 6 7]
arr_slice[1] = 12345
print(arr)                # [    0    1    2    3    4    5 12345    7    8    9]
arr_slice[:] = 64
print(arr)                # [ 0  1  2  3  4 64 64 64  8  9]
```

Eixos

```
a.sum() # soma todas as células

a.sum(axis=0) # soma sobre as linhas

a.sum(axis=1) # soma sobre as colunas

a.sum(axis=1, keepdims=True)
```

1. Utilize o parâmetro **axis** para controlar qual eixo o Numpy irá operar, indicando qual eixo será reduzido ou eliminado na operação
2. Normalmente o eixo especificado vai desaparecer, usar o **keepdims** mantem todas as dimensões

Eixos

Exemplo:

```
a = np.array([    [1, 2, 3],    [4, 5, 6]])
# (2, 3)
#  ↑  ↑
#  |  └── axis=1 → colunas
#  └───── axis=0 → linhas

a.sum(axis=0) # [5 7 9]
# O eixo 0 desapareceu. Para cada coluna, os valores das linhas foram somados.

a.sum(axis=1)# [ 6 15]
# O eixo 1 desapareceu. Para cada linha, os valores das colunas foram somados.
```

Agregações

Funções de agregação no numpy: **mínimo**, **máximo**, **media**, **mediana**, **desvio padrão**

```
dados = np.array([5, 7, 8, 10, 10])

dados.sum()

dados.mean()

dados.min()

dados.max()

np.median(dados)

dados.std()
```

```
>>> dados = np.array([5, 7, 8, 10, 10])
>>> dados.sum()
np.int64(40)
>>> dados.mean()
np.float64(8.0)
>>> dados.min()
np.int64(5)
>>> dados.max()
np.int64(10)
>>> np.median(dados)
np.float64(8.0)
>>> dados.std()
np.float64(1.8973665961010275)
>>> notas = np.array([
...     [7, 8, 9],
...     [5, 6, 7],
...     [9, 10, 8]
... ])
>>> notas.mean(axis=1)
array([8., 6., 9.])
```

Dicas para evitar problemas

1. Saiba quais são seus tipos de dados.
2. Verifique se você tem uma visualização ou uma cópia.
3. Use **matplotlib** para verificações de integridade.
4. Use o debug para verificar cada etapa do seu cálculo.
5. Conheça a diferença entre **np.multiply** e **np.matmul**
   - A * B ou np.multiply(A, B)
   - A @ B ou np.matmul(A, B)

Exercícios

Dada a matriz:

```
frequencias = np.array([
    [3, 0, 1, 2],
    [0, 2, 4, 1],
    [1, 1, 0, 5]
])
```

E sabendo que:

- Cada linha corresponde a um documento
- Cada coluna corresponde a uma palavra
- Cada célula corresponde a quantidade de ocorrências

Informe:

1. Qual o shape da matriz?
2. Quantos documentos existem?
3. Quantas palavras existem no vocabulário?
4. Some a frequência total de cada palavra.
5. Some a quantidade de palavras de cada documento.
6. Identifique quais valores são maiores que 2.